Interveil — Full Phase Instruction Specification

Purpose: Feed each phase to an AI coding agent as implementation instructions.

Each phase is self-contained and exhaustive.

Product name: Interveil

All references to “AgentTrace” have been replaced with “Interveil”

PHASE 1 — Core Observability (MVP)

Guiding Principle

Interveil is a **language-agnostic observability server**. The core product is an HTTP server that accepts trace events from any agent written in any language. Language-specific SDKs (TypeScript, Python, etc.) are thin convenience wrappers that call this server. A developer using Go, Rust, Java, or any other language can integrate by making plain HTTP requests — no SDK required.

Build only what is described in this phase. Do not anticipate future phases. Do not add features not listed here.

Architecture Overview

```
Agent (any language)
        |
        |  HTTP POST /api/v1/events
        ▼
```

```
Interveil Server (TypeScript / Node.js)
        |
        |  WebSocket broadcast
        ▼
Trace UI (browser, localhost:3000)
```

The server is the product. The SDKs are optional convenience layers.

Package Identity

- Package name: `interveil`
- Published to: npm registry
- Global install command: `npm install -g interveil`
- CLI binary name: `interveil`
- Language: TypeScript
- Runtime: Node.js 18+
- Entry point: `src/index.ts`
- Exports: `trace()` wrapper function, `InterveillClient` class

HTTP API — Trace Ingestion

This is the core integration surface. Every SDK and every direct integration uses these endpoints.

`POST /api/v1/events`

Ingest a single trace event.

Request body:

```
{
  "session_id": "string (required) — uuid v4 identifying the agent run",
  "step_index": "integer (required) — zero-based order of this step in the run",
  "step_type": "string (required) — see Step Types below",
  "input": "object (required, nullable) — full input payload for this step",
  "output": "object (required, nullable) — full output payload for this step",
```

```
  "timestamp": "string (required) — ISO 8601 with milliseconds e.g. 2025-04-27T14
  "duration_ms": "integer (optional) — how long this step took",
  "token_usage": {
    "prompt_tokens": "integer",
    "completion_tokens": "integer",
    "total_tokens": "integer"
  },
  "model": "string (optional) — model name e.g. claude-sonnet-4-20250514",
  "error": "object (optional, nullable) — error details if step threw",
  "metadata": "object (optional) — any additional key-value pairs"
}
```

Response `200 OK`:

```
{
  "ok": true,
  "event_id": "uuid of stored event"
}
```

Response `400 Bad Request`:

```
{
  "ok": false,
  "error": "human readable validation error"
}
```

`POST /api/v1/sessions`

Register a new session before sending events. Optional but recommended.

Request body:

```
{
  "session_id": "string (required) — uuid v4, generated by the client",
  "name": "string (optional) — human readable e.g. 'user-onboarding-run-1'",
  "agent_id": "string (optional) — identifier for the agent type",
  "metadata": "object (optional)"
}
```

Response `200 OK`:

```
{
  "ok": true,
```

```
  "session_id": "string"
}
```

`PATCH /api/v1/sessions/:session_id`

Update session status when run completes or fails.

Request body:

```
{
  "status": "completed | failed",
  "ended_at": "ISO 8601 timestamp"
}
```

Response `200 OK`:

```
{ "ok": true }
```

`GET /api/v1/health`

Health check endpoint. Returns 200 if server is running. Used by SDKs and the VSCode extension to detect whether the server is active.

Response `200 OK`:

```
{
  "ok": true,
  "version": "package version string"
}
```

Step Types

The `step_type` field must be one of the following string values:

Value	Meaning
`REASONING`	Internal agent thought, scratchpad, or chain-of-thought output
`TOOL_CALL`	Agent invoking a tool or function

`TOOL_RESULT`	Result returned from a tool invocation
`LLM_REQUEST`	Full prompt sent to a language model
`LLM_RESPONSE`	Full response received from a language model
`ERROR`	An exception or failure occurred at this step
`CUSTOM`	Any event type the developer wants to emit manually

WebSocket — Live Streaming

The server exposes a WebSocket endpoint at `ws://localhost:3000/ws`.

- The browser UI connects to this endpoint on page load
- Every accepted `POST /api/v1/events` request immediately broadcasts the full event payload to all connected WebSocket clients
- WebSocket message format is identical to the POST request body, with `event_id` appended
- No authentication required in Phase 1
- If WebSocket connection drops, the UI attempts reconnect every 2 seconds

---

TypeScript SDK — Convenience Wrapper

The TypeScript SDK is a thin client that handles session creation, event formatting, and HTTP posting. It does not contain any core logic — it is purely a developer experience layer on top of the HTTP API.

Installation

```
npm install interveil
```

One-Line Wrapper API

```
import { trace } from 'interveil'
```

```
const tracedAgent = trace(yourExistingAgent, options?)
```

What `trace()` Does Internally

1. Generates a `session_id` (uuid v4)
2. POSTs to `POST /api/v1/sessions` with optional name from options
3. Wraps the agent object in a JavaScript `Proxy`
4. The Proxy intercepts every method call on the agent
5. For each intercepted call, before executing the real method:
   - Records `timestamp` and starts a timer
   - Serializes the method name and arguments as `input`
6. After the real method resolves or throws:
   - Records `duration_ms`
   - Serializes the return value as `output` or error as `error`
   - Determines `step_type` based on method name heuristics (see below)
   - POSTs the event to `POST /api/v1/events`
7. Returns the real method's result unmodified — agent behavior is never changed
8. On process exit or explicit `Interveil.end()` call, PATCHes session status to `completed` or `failed`

Step Type Heuristics

The TypeScript SDK infers `step_type` from method names using these rules (in order):

- Method name contains `think`, `reason`, `plan`, `reflect` → `REASONING`
- Method name contains `call`, `invoke`, `run`, `execute`, `use` → `TOOL_CALL`
- Method name is the return handler of a tool call → `TOOL_RESULT`
- Method name contains `complete`, `chat`, `message`, `generate`, `predict` → `LLM_REQUEST` (before) and `LLM_RESPONSE` (after)
- Method threw an exception → `ERROR`
- Anything else → `CUSTOM`

Developer can override by passing explicit `stepType` in metadata.

Options Object

```
interface TraceOptions {
  port?: number           // default: 3000 — port Interveil server is running on
  host?: string           // default: 'localhost'
  autoOpen?: boolean      // default: true — auto-opens browser tab on first even
  sessionName?: string    // default: ISO timestamp
  agentId?: string        // default: undefined
  verbose?: boolean       // default: false — logs each event to console as it is
}
```

Auto-Open Browser

- When `autoOpen: true` (default), the SDK calls `open('http://localhost:3000')` after the first event is successfully posted
- Uses the `open` npm package for cross-platform browser launching
- Only opens once per process — not on every event

Manual Event API

```
import { InterveillClient } from 'interveil'

const client = new InterveillClient({ port: 3000 })
const session = await client.startSession({ name: 'my-run' })

await client.emit({
  session_id: session.session_id,
  step_index: 0,
  step_type: 'REASONING',
  input: { prompt: 'What should I do next?' },
  output: { thought: 'I should check the database first' },
  timestamp: new Date().toISOString(),
  duration_ms: 45
})

await client.endSession(session.session_id, 'completed')
```

Python SDK — Thin Client

Installation

```
pip install interveil
```

Usage

```
from interveil import InterveillClient

client = InterveillClient(port=3000)
session = client.start_session(name="my-python-agent-run")

client.emit(
    session_id=session["session_id"],
    step_index=0,
    step_type="REASONING",
    input={"prompt": "What should I do?"},
    output={"thought": "Check the database first"},
    timestamp=datetime.utcnow().isoformat() + "Z",
    duration_ms=45
)

client.end_session(session["session_id"], "completed")
```

Python SDK Implementation Requirements

- Pure Python, no dependencies beyond `requests`
- `InterveillClient.__init__` accepts `host` (default `localhost`) and `port` (default `3000`)
- All methods map 1:1 to the HTTP API endpoints described above
- `emit()` is synchronous by default, async version `emit_async()` available using `aiohttp`
- If the Interveil server is not running, all methods fail silently with a console warning — they must never raise an exception that crashes the developer's agent

---

Direct HTTP Integration (No SDK)

For any language without an SDK, developers can integrate by making plain HTTP requests.

Example: curl

```
# Start a session
curl -X POST http://localhost:3000/api/v1/sessions \
  -H "Content-Type: application/json" \
  -d '{"session_id": "abc-123", "name": "my-run"}'

# Send an event
curl -X POST http://localhost:3000/api/v1/events \
  -H "Content-Type: application/json" \
  -d '{
    "session_id": "abc-123",
    "step_index": 0,
    "step_type": "TOOL_CALL",
    "input": {"tool": "search", "query": "latest AI news"},
    "output": {"results": ["..."]},
    "timestamp": "2025-04-27T14:32:01.123Z",
    "duration_ms": 340
  }'
```

Document this pattern prominently in the README — it is a first-class integration path.

Browser UI — Trace Viewer

The UI is a single-page application served by the Interveil server at `http://localhost:3000`. It has zero external dependencies — all assets are bundled into the npm package and served statically.

Layout

- Dark theme (background `#0F172A`, surface `#1E293B`, text `#F1F5F9`)
- Top bar: Interveil logo, session name, live/disconnected status indicator, step count, total duration
- Main panel: chronological list of trace steps (~65% width)
- Detail panel: full JSON view of selected step (~35% width, hidden by default, slides in on click)

Step List

Each step renders as a row containing:

- **Step type badge** with color coding:
  - `REASONING` → blue `#3B82F6`
  - `TOOL_CALL` → orange `#F97316`
  - `TOOL_RESULT` → green `#22C55E`
  - `LLM_REQUEST` → purple `#A855F7`
  - `LLM_RESPONSE` → teal `#14B8A6`
  - `ERROR` → red `#EF4444`
  - `CUSTOM` → gray `#6B7280`
- Step index — small gray number
- Timestamp — relative to session start e.g. "+2.3s"
- Duration badge — e.g. "340ms"
- Summary — first 120 characters of output serialized to string, truncated with ellipsis
- Entire row is clickable — opens detail panel

Detail Panel

When a step is selected:

- Shows full `input` as formatted, syntax-highlighted JSON
- Shows full `output` as formatted, syntax-highlighted JSON
- Shows full `error` object if present
- Shows all metadata fields
- Shows `model` and `token_usage` if present
- Copy button copies the full event as JSON to clipboard

Failure Highlighting

- The step immediately preceding any `ERROR` step gets a yellow left border `#EAB308` and subtle yellow background tint
- `ERROR` steps render with red left border, red background tint, and are expanded by default

- A dismissible banner at the top of the step list: `"⚠ Failure detected at step {N} — {error.message}"` in red

Live Status Indicator

- Pulsing green dot + text `"Live"` when WebSocket is connected
- Static red dot + text `"Disconnected — retrying..."` when WebSocket drops
- Automatically reconnects and resumes live updates when server comes back

Empty State

- Center of screen: `"Waiting for your agent..."`
- Below: `"Run your agent to see its trace here"`

In-Memory Store (Phase 1 Only)

Phase 1 uses no database. All sessions and events are stored in memory.

```
const sessions: Map<string, Session> = new Map()
const events: Map<string, TraceEvent[]> = new Map()  // keyed by session_id
```

- On server restart all data is lost — acceptable in Phase 1
- Store is implemented behind an interface so Phase 4 can swap in SQLite without changing any other code

```
interface TraceStore {
  createSession(session: Session): Promise<void>
  updateSession(id: string, update: Partial<Session>): Promise<void>
  addEvent(event: TraceEvent): Promise<void>
  getSession(id: string): Promise<Session | null>
  getEvents(sessionId: string): Promise<TraceEvent[]>
}
```

File Structure

```
interveil/
├── src/
```

```
│   ├── index.ts              # exports trace(), InterveillClient
│   ├── server.ts             # Express HTTP server + WebSocket server
│   ├── cli.ts                # CLI binary entry point
│   ├── api/
│   │   ├── events.ts         # POST /api/v1/events handler
│   │   ├── sessions.ts       # POST + PATCH /api/v1/sessions handler
│   │   └── health.ts         # GET /api/v1/health handler
│   ├── sdk/
│   │   ├── trace.ts          # trace() wrapper, Proxy logic
│   │   └── client.ts         # InterveillClient class
│   ├── store/
│   │   └── memory.ts         # in-memory event store (Phase 1 only, no DB)
│   ├── ws/
│   │   └── broadcaster.ts    # WebSocket broadcast logic
│   └── ui/
│       ├── index.html        # single-page UI shell
│       ├── app.js            # UI logic, WebSocket client, rendering
│       └── styles.css        # UI styles, dark theme
├── python/
│   ├── interveil/
│   │   ├── __init__.py
│   │   └── client.py         # InterveillClient Python class
│   ├── pyproject.toml
│   └── README.md
├── examples/
│   ├── typescript/
│   │   ├── basic-agent.ts
│   │   └── README.md
│   └── python/
│       ├── basic_agent.py
│       └── README.md
├── package.json
├── tsconfig.json
└── README.md
```

CLI Binary

The `interveil` global binary exposes one command in Phase 1:

```
interveil serve [--port 3000] [--verbose]
```

- Starts the HTTP + WebSocket server
- Prints: `Interveil running at http://localhost:3000`

- `--verbose` flag logs every incoming event to console

Non-Functional Requirements

- Server must handle at least 100 concurrent WebSocket connections without degradation
- `POST /api/v1/events` must respond in under 5ms (in-memory store, no DB in Phase 1)
- All HTTP responses must include `Content-Type: application/json`
- Server must not crash on malformed JSON — return 400 with descriptive error
- Server must not crash if a WebSocket client disconnects unexpectedly
- TypeScript SDK must never throw an unhandled exception — all errors caught internally and logged with `[Interveil]` prefix
- Python SDK must never raise an exception that propagates to the developer's code

---

Exit Criteria

1. `npm install -g interveil && interveil serve` starts a server with no errors
2. A TypeScript agent wrapped with `trace()` auto-opens the browser and streams all steps in real time
3. A Python agent using `InterveillClient` streams all steps to the same UI
4. A raw `curl` POST to `/api/v1/events` appears in the UI within 100ms
5. When an agent throws an error, the failure is highlighted and the preceding step has a yellow border
6. The server handles a 30-step agent run without crashing or missing events
7. Stopping and restarting the server works cleanly with no port conflicts

---

PHASE 1.1 — Containerization + Publishing

Overview

Make Interveil runnable via Docker for team environments and production use cases where global npm installs are undesirable. Formally publish the npm package and create a demo environment.

Docker Requirements

- Provide a `Dockerfile` in the root of the repository
- Provide a `docker-compose.yml` in the root of the repository
- Running `docker-compose up` must:
  - Build the Interveil image
  - Start the trace server on port 3000
  - Mount a local volume at `/traces` for persistent trace storage (forward-compatible with Phase 4)
  - Print a success message with the local URL to the console

Dockerfile Specification

- Base image: `node:18-alpine`
- Working directory: `/app`
- Copy package files and install dependencies
- Copy source and build TypeScript
- Expose port 3000
- Default command starts the Interveil server in listen mode

docker-compose.yml Specification

```
services:
  interveil:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - ./traces:/app/traces
    environment:
      - NODE_ENV=production
```

npm Publishing

- Ensure `package.json` has correct `main`, `types`, and `bin` fields
- `bin.interveil` points to the CLI entry point
- `files` array includes only `dist/` and `README.md`
- Run `npm publish --access public`
- Package must install cleanly on macOS, Linux, and Windows (WSL)

Demo Environment

- Create a `/examples` directory in the repository
- Include one working example: `examples/typescript/basic-agent.ts`
- This example must:
  - Instantiate a simple AI agent using the OpenAI SDK or Anthropic SDK (developer's choice)
  - Wrap it with `trace()`
  - Execute a task that involves at least 3 tool calls
  - Intentionally include one step that fails so failure highlighting is visible
- Include `examples/typescript/README.md` explaining how to run it

PHASE 1.5 — Interveil Gateway (LLM Proxy + MCP Proxy)

Overview

This is load-bearing infrastructure. The Gateway allows Interveil to observe and intercept **any** AI agent — including black-box agents built into IDEs like Cursor, Continue.dev, and Claude Code — without requiring the developer to wrap their agent in an SDK or modify any code. The developer simply points their existing tool at Interveil instead of directly at the LLM provider.

The Gateway is required for Phases 2 and 3 to function for black-box agent users. Without

it, command control and access control only work for developers using the SDK directly.

Build only what is described in this phase. Do not anticipate future phases. Do not add features not listed here.

Architecture Overview

```
IDE / Agent Tool (Cursor, Claude Code, Continue.dev, etc.)
        |
        |  HTTP (OpenAI-compatible API)
        ▼
Interveil Gateway  ← sits here as a transparent proxy
        |                    |
        |  logs to Trace UI  |  forwards request
        ▼                    ▼
Trace Viewer (browser)    Real LLM Provider (OpenAI / Anthropic / etc.)
```

The user configures their IDE or agent tool to use `http://localhost:3000/v1/proxy` as the base URL instead of `https://api.openai.com`. Interveil intercepts every request, logs it as a trace event, forwards it to the real provider, receives the response, logs it, and passes it back to the IDE. From the IDE's perspective, Interveil is invisible.

LLM Proxy — OpenAI-Compatible Endpoint

Why OpenAI-Compatible

The OpenAI API has become the de facto standard. Cursor, Continue.dev, Aider, and most agentic tools support a configurable base URL for the OpenAI API. By implementing the OpenAI spec, Interveil becomes compatible with all of them simultaneously. Anthropic, Mistral, and other providers can be proxied by translating their requests/responses to/from the OpenAI format.

Endpoint: `POST /v1/proxy/chat/completions`

This endpoint mirrors the OpenAI `POST /v1/chat/completions` specification exactly.

Behavior:

1. Receive the incoming request from the IDE/agent
2. Extract and log a `LLM_REQUEST` trace event containing:

- `model` : from request body
- `messages` : full messages array from request body
- `tools` : tool definitions if present
- `temperature` , `max_tokens` , and any other parameters
- `source` : `"gateway"`
- `timestamp` : ISO 8601

3. Determine the real provider from the `model` field:
   - Model starts with `gpt-` or `o1` or `o3` → forward to `https://api.openai.com/v1/chat/completions`
   - Model starts with `claude-` → translate to Anthropic format, forward to `https://api.anthropic.com/v1/messages` , translate response back
   - Model starts with `gemini-` → translate to Google format, forward, translate back
   - Unknown model → forward to OpenAI by default
4. Forward the request to the real provider using the user's API key (see Auth Passthrough below)
5. Receive the response from the real provider
6. Log a `LLM_RESPONSE` trace event containing:
   - Full response body
   - `duration_ms` : total round-trip time
   - `token_usage` : from response if available
   - `finish_reason` : from response
7. Return the response to the IDE/agent unmodified

Streaming Support (Server-Sent Events)

This is critical. Most IDEs use streaming to show the "typewriter" effect as the model responds. Interveil must support streaming transparently.

Streaming behavior:

- Detect if the incoming request has `"stream": true`

- If streaming: open a streaming connection to the real provider
- As each SSE chunk arrives from the real provider:
  - Immediately forward it to the IDE (do not buffer)
  - Accumulate the full response in memory
- After the stream ends (`data: [DONE]`):
  - Emit the completed `LLM_RESPONSE` trace event with the fully assembled response
- Latency added by streaming proxy must be under 10ms per chunk

Auth Passthrough

The developer's API key must never be stored by Interveil. Two modes:

Mode 1 — Passthrough (default):

- The IDE sends its API key in the `Authorization: Bearer {key}` header as normal
- Interveil extracts the key, uses it to call the real provider, and discards it
- Interveil never logs or stores the API key

Mode 2 — Stored key (optional):

- Developer sets environment variables: `OPENAI_API_KEY` , `ANTHROPIC_API_KEY` , etc.
- Interveil uses these instead of requiring the IDE to pass a key
- Useful when the IDE does not support custom auth headers

Both modes must work. Mode 1 is default. If neither a passthrough key nor an environment variable key is available, Interveil returns a `401` with a helpful error message explaining how to configure the key.

Additional OpenAI-Compatible Endpoints

Implement these stubs so IDEs that check for them do not error:

- `GET /v1/proxy/models` — returns a static list of common model names in OpenAI format
- `POST /v1/proxy/embeddings` — proxies to real embeddings endpoint, logs as `CUSTOM` trace event
- `GET /v1/proxy/health` — returns `{ "ok": true }`

MCP Proxy

What is MCP

The Model Context Protocol (MCP) is an open standard that allows AI agents to call tools (like “edit file”, “run terminal command”, “search the web”) via a standardized interface. Agents that support MCP call tools by sending MCP-formatted requests to an MCP server.

How the MCP Proxy Works

Interveil sits between the agent and the real MCP tool executor:

```
Agent
  │  MCP tool call request
  ▼
Interveil MCP Proxy  ← logs the call before it executes
  │
  │  forwards to real MCP server
  ▼
Real MCP Tool Executor (filesystem, terminal, browser, etc.)
  │
  │  result
  ▼
Interveil MCP Proxy  ← logs the result
  │
  ▼
Agent
```

MCP Proxy Endpoint: `POST /v1/mcp`

Receives MCP tool call requests from the agent.

Request body (MCP format):

```
{
  "method": "tools/call",
  "params": {
    "name": "edit_file",
    "arguments": {
      "path": "/src/index.ts",
      "content": "..."
    }
  }
}
```

Behavior:

1. Receive MCP tool call request
2. Log a `TOOL_CALL` trace event containing:
   - `tool_name` : from `params.name`
   - `arguments` : from `params.arguments`
   - `protocol` : `"mcp"`
   - `timestamp` : ISO 8601
3. Forward the request to the configured real MCP server (`MCP_SERVER_URL` environment variable)
4. Receive the result from the real MCP server
5. Log a `TOOL_RESULT` trace event containing the full result
6. Return the result to the agent

MCP Configuration

Developer configures Interveil with their real MCP server URL:

```
interveil serve --mcp-server http://localhost:8080
```

Or via environment variable:

```
MCP_SERVER_URL=http://localhost:8080 interveil serve
```

If no MCP server is configured, the MCP proxy endpoint returns a `503` with a helpful message.

Gateway Configuration UI

Add a new “Gateway” tab to the Interveil browser UI at `http://localhost:3000` .

Gateway Tab Contents

- **LLM Proxy Status**: shows whether the proxy is active, what port it is on
- **Setup Instructions**: step-by-step instructions for 3 specific IDE integrations:

- **Cursor**: Settings → Models → Base URL → `http://localhost:3000/v1/proxy`
- **Continue.dev**: `config.json` → `"apiBase": "http://localhost:3000/v1/proxy"`
- **Generic (any OpenAI-compatible tool)**: Set base URL to `http://localhost:3000/v1/proxy`

- **MCP Proxy Status**: shows whether MCP proxy is configured, what server it is pointing to
- **API Key Mode**: shows whether using passthrough or stored key mode
- **Live Request Log**: last 10 requests proxied, showing model, token count, latency

---

Session Correlation

Gateway requests must be linked to a trace session so they appear in the trace viewer alongside SDK-emitted events.

- When the gateway receives a request, it checks for an `X-Interveil-Session-Id` header
- If present: all trace events from this request are attributed to that session
- If absent: the gateway creates a new session automatically, named `"Gateway — {model} — {timestamp}"`
- The session appears in the trace viewer immediately, labeled with a `GATEWAY` badge

---

Error Handling

The gateway must handle these failure cases gracefully:

Failure	Behavior
Real LLM provider returns 4xx	Pass through the error to the IDE unchanged. Log an `ERROR` trace event.
Real LLM provider returns 5xx	Pass through unchanged. Log `ERROR`.
Real LLM provider unreachable	Return `502 Bad Gateway` to IDE with message: `"Interveil Gateway: could not reach provider"`

Invalid API key	Pass through 401 from provider. Log `ERROR` with message about key configuration.
MCP server unreachable	Return `503` to agent. Log `ERROR`.
Streaming connection dropped mid-stream	Close connection to IDE, log partial `LLM_RESPONSE` event with `"partial": true` flag

---

File Structure Additions for Phase 1.5

```
interveil/
├── src/
│   ├── gateway/
│   │   ├── llm-proxy.ts        # POST /v1/proxy/chat/completions handler
│   │   ├── mcp-proxy.ts        # POST /v1/mcp handler
│   │   ├── streaming.ts        # SSE streaming proxy logic
│   │   ├── auth.ts             # API key passthrough and stored key logic
│   │   ├── providers/
│   │   │   ├── openai.ts       # OpenAI format — passthrough
│   │   │   ├── anthropic.ts    # Anthropic format — request/response translation
│   │   │   └── gemini.ts       # Google Gemini format — request/response transla
│   │   └── models.ts           # GET /v1/proxy/models static response
│   └── ui/
│       └── gateway-tab.js      # Gateway tab UI logic
```

CLI Updates for Phase 1.5

```
interveil serve [--port 3000] [--verbose] [--mcp-server <url>]
```

New flag: `--mcp-server <url>` — configures the MCP proxy target

Non-Functional Requirements

- Streaming proxy latency overhead must be under 10ms per chunk
- Non-streaming proxy latency overhead must be under 20ms total
- Gateway must handle at least 10 simultaneous streaming connections

- API keys must never appear in logs, trace events, or the UI — not even partially
- Gateway must work correctly when Interveil server is running inside Docker (host networking considerations must be handled)

Exit Criteria

1. Cursor (or any OpenAI-compatible tool) configured with `http://localhost:3000/v1/proxy` as base URL successfully completes a chat completion — response is correct and identical to calling OpenAI directly
2. A streaming request shows the typewriter effect in the IDE with no visible latency increase
3. The completed request appears in the Interveil trace UI as a `LLM_REQUEST` + `LLM_RESPONSE` pair
4. An MCP tool call proxied through `/v1/mcp` appears in the trace UI as `TOOL_CALL` + `TOOL_RESULT`
5. The Gateway tab in the UI shows correct setup instructions and live request log
6. A request made without any API key returns a clear, actionable error message
7. If the real LLM provider is unreachable, the IDE receives a clean `502` error and Interveil does not crash

---

PHASE 1.75 — VSCode Extension

Overview

Build and publish a VSCode extension that renders the Interveil trace viewer as a native VSCode side panel, eliminating the need for a separate browser tab.

Extension Identity

- Extension name: `interveil-vscode`
- Display name: `Interveil`
- Published to: VSCode Marketplace via `vsce`

Activation

- Extension activates when a TypeScript or JavaScript file is open
- Extension activates when `interveil` package is detected in the workspace's `node_modules`
- Adds a new icon to the VSCode Activity Bar (left sidebar icon strip)

Side Panel Behavior

- Clicking the Activity Bar icon opens the Interveil side panel
- Side panel contains a WebView that loads `http://localhost:3000`
- Panel auto-refreshes its connection when the port changes
- If Interveil is not running, panel shows: "Start Interveil to see traces here" with installation docs link
- Panel persists across file switches

Port Detection

- Extension reads the `interveil.port` VSCode workspace setting (default: 3000)
- If default port unavailable, polls 3000–3010 and connects to first valid health check response

VSCode Settings Contribution

```
{
  "interveil.port": {
    "type": "number",
    "default": 3000,
    "description": "Port that the Interveil server is running on"
  },
  "interveil.autoConnect": {
    "type": "boolean",
    "default": true,
    "description": "Automatically connect to Interveil when detected"
  }
}
```

Publishing Process

- Install `vsce`: `npm install -g @vscode/vsce`
- Run `vsce package` to produce `.vsix` file
- Run `vsce publish` to publish to marketplace

PHASE 2 — Command Control

Overview

Intercept and evaluate every command or shell action an agent attempts to execute before it runs. This phase works for **both** SDK-wrapped agents (via the TypeScript Proxy) and black-box IDE agents (via the Gateway MCP proxy from Phase 1.5). Block dangerous commands based on a configurable ruleset. Log all blocked attempts to the trace view. Provide a dry-run mode for safe simulation.

Core Concepts

- A "command" is any shell execution, subprocess spawn, or OS-level action the agent triggers
- For SDK users: Interveil wraps Node.js `child_process` methods at the module level
- For Gateway/MCP users: command interception happens at the MCP proxy layer — tool calls with dangerous arguments are blocked before being forwarded to the real MCP server
- Interception happens before execution — the command is evaluated against the ruleset, then either allowed, blocked, or flagged

Blocklist

- Interveil ships with a built-in default blocklist of dangerous command patterns
- Default blocklist must include at minimum:
  - `rm -rf` and variants (`rm -r` , `rm --recursive`)
  - `DROP TABLE` , `DROP DATABASE` , `TRUNCATE` (case-insensitive SQL)

    - `FORMAT`, `DISKPART` (Windows destructive commands)
    - `chmod 777` on system directories
    - `curl | bash` and `wget | bash` patterns (arbitrary remote execution)
    - `sudo rm`, `sudo mkfs`
    - Any command targeting `/etc/passwd`, `/etc/shadow`, `~/.ssh/`
- Blocklist stored as array of strings and RegExp patterns in `config/blocklist.json`

User Configuration

Developer creates `interveil.config.json` in their project root:

```
{
  "commands": {
    "block": ["string or regex pattern"],
    "whitelist": ["string or regex pattern"],
    "requireApproval": ["string or regex pattern"]
  }
}
```

- `block` : additional patterns beyond defaults
- `whitelist` : patterns that override the default blocklist
- `requireApproval` : patterns that pause execution and prompt developer for y/n in terminal

Block Event Behavior

When a command is blocked:

- Command does not execute
- A `COMMAND_BLOCKED` trace event is emitted containing:
    - `blocked_command` : full command string attempted
    - `matched_rule` : specific rule or pattern that triggered the block
    - `rule_source` : `default_blocklist | user_config | approval_required`
    - `timestamp` : ISO 8601

- Trace UI renders this event with a red shield icon
- Agent receives error response: `"Command blocked by Interveil: [matched_rule]"`

Dry-Run Mode

- Activated by `{ dryRun: true }` in `trace()` options or `"dryRun": true` in `interveil.config.json`
- In dry-run mode, NO commands execute regardless of blocklist rules
- Every command attempt emits a `COMMAND_DRY_RUN` trace event showing what would have executed
- Agent receives mock success response for all commands
- Dry-run mode shown in UI with yellow "DRY RUN" banner at top

Webhook Support

- Optional config key `"webhooks"` accepts array of URLs
- On every `COMMAND_BLOCKED` event, Interveil POSTs to each webhook URL with full block event payload
- Payload includes `source: "interveil"` field
- Supports Slack incoming webhooks natively

PHASE 3 — Tool Registry + Access Control

Overview

Allow developers to formally register their agent's tools with Interveil using Zod schemas, then define granular access control rules per tool per agent. All tool calls are intercepted at the wrapper level — making it impossible for an agent to bypass controls through prompt injection or indirect invocation. For Gateway users, tool access control is enforced at the MCP proxy layer.

Tool Registration API

```
import { registerTool } from 'interveil'

registerTool({
  name: 'database_query',
  description: 'Executes a SQL query against the application database',
  schema: z.object({
    query: z.string(),
    database: z.enum(['production', 'staging', 'test']),
    readonly: z.boolean()
  }),
  handler: async (input) => { /* actual tool implementation */ }
})
```

Access Control Rules

```
import { definePolicy } from 'interveil'

definePolicy({
  agentId: 'research-agent',
  rules: [
    {
      tool: 'database_query',
      allow: ['read'],
      deny: ['write', 'execute'],
      conditions: {
        'input.database': { mustEqual: 'staging' }
      }
    },
    {
      tool: 'file_system',
      allow: ['read'],
      deny: ['write', 'delete']
    },
    {
      tool: 'external_api',
      allow: [],
      deny: ['*']
    }
  ]
})
```

Permission Types

- `read` — tool may retrieve data but not modify

- `write` — tool may create or update data
- `execute` — tool may run processes or trigger side effects
- `delete` — tool may remove data
- `*` — wildcard matching all permission types

Condition Evaluation

- Conditions evaluated against tool's input payload at call time
- Supported operators: `mustEqual`, `mustNotEqual`, `mustContain`, `mustMatch` (regex), `mustBeOneOf`
- If any condition fails, call is treated as a permission violation

Interception Architecture

Access control must be enforced at the execution layer, not just the registration layer.

- For SDK users: Interveil's Proxy intercepts every method call before execution
- For Gateway/MCP users: Interveil's MCP proxy checks every tool call against the policy before forwarding to the real MCP server
- There is no path to execute a registered tool that bypasses this interception
- Unregistered tool calls are logged as `TOOL_UNREGISTERED` warnings but allowed by default (configurable)

Permission Violation Trace Event

```
{
  step_type: 'TOOL_PERMISSION_DENIED',
  tool_name: string,
  agent_id: string,
  attempted_permissions: string[],
  allowed_permissions: string[],
  violated_rule: string,
  input_payload: object,
  timestamp: string
}
```

Policy Export

- `exportPolicy()` serializes current policy to JSON file for version control
- `importPolicy(filePath)` loads policy from JSON file at startup

PHASE 4 — Memory + History

Overview

Persist all trace data beyond the current process lifetime. Every agent run is stored, searchable, and comparable.

Storage Layer

- Default storage: SQLite database at `~/.interveil/traces.db`
- Configurable via `interveil.config.json` key `"storagePath"`
- All writes are async and non-blocking — storage must never slow down agent execution
- Replaces the in-memory store from Phase 1 behind the same `TraceStore` interface

Database Schema (Core Tables)

sessions

```
id TEXT PRIMARY KEY,
name TEXT,
agent_id TEXT,
started_at TEXT,
ended_at TEXT,
status TEXT,         -- 'running' | 'completed' | 'failed'
tags TEXT,           -- JSON array
metadata TEXT        -- JSON object
```

trace_events

```
id TEXT PRIMARY KEY,
session_id TEXT REFERENCES sessions(id),
step_index INTEGER,
step_type TEXT,
```

```
input TEXT,
output TEXT,
duration_ms INTEGER,
token_usage TEXT,
model TEXT,
error TEXT,
timestamp TEXT
```

labels

```
id TEXT PRIMARY KEY,
session_id TEXT REFERENCES sessions(id),
label TEXT,
created_at TEXT,
notes TEXT
```

Session Management API

```
trace(agent, { sessionName: 'user-onboarding-flow-v2' })
await Interveil.tagSession(sessionId, ['regression', 'critical-path'])
await Interveil.labelSession(sessionId, 'failure', 'Hallucinated database name')
```

Search + Filter UI

History tab in trace viewer:

- Full-text search across session names and trace event content
- Filter by date range, status, label, agent_id, tag
- Session list sorted by most recent

Diff View

- Select two sessions, click “Compare”
- Side-by-side aligned by `step_index`
- Color-coded diffs: yellow (changed), red (missing), green (added)
- Token usage and duration deltas shown at top

Failure Library

- Sessions labeled `failure` or `edge_case` appear in “Failure Library” tab
- Export as JSONL for use as eval dataset

Cost Tracking

- Every `LLM_REQUEST` event with token_usage contributes to cost tracking
- Model pricing table in `config/pricing.json`, user-updatable
- UI shows spend per session, today, this week, this month, per model

---

PHASE 5 — Sidecar Monitor

Overview

Run a second, context-clean AI model in parallel with the primary agent. This model only observes the trace stream — it never sees the agent’s own context window. Its job is to detect anomalies, explain failures, and optionally halt the agent.

Architecture

- When `trace(agent, { sidecar: true })` is passed, Interveil spawns a sidecar process
- Sidecar runs in a completely separate Node.js process
- Communication via IPC only — sidecar never accesses agent’s internal state
- Works for both SDK users and Gateway users (sidecar reads trace events regardless of source)

Sidecar Configuration

```
trace(agent, {
  sidecar: {
    enabled: true,
    model: 'claude-sonnet-4-20250514',
    interventionLevel: 'warn',   // 'observe' | 'warn' | 'halt'
    systemPrompt: '...'
  }
})
```

Anomaly Types

- **Loop detection**: same tool called with identical inputs 3+ times without meaningful output change
- **Contradiction**: agent reasoning in step N contradicts conclusion in step M
- **Unexpected tool usage**: tool called with no clear relationship to stated goal
- **Permission boundary probing**: multiple near-miss permission violations in sequence
- **Context drift**: agent's stated goal in later steps diverges from initial instruction

Anomaly Event Schema

```
{
  step_type: 'SIDECAR_ANOMALY',
  anomaly_type: 'loop' | 'contradiction' | 'unexpected_tool' | 'permission_probe'
  severity: 'low' | 'medium' | 'high' | 'critical',
  affected_steps: string[],
  explanation: string,
  recommendation: string,
  confidence: number    // 0.0 - 1.0
}
```

Intervention Levels

- `observe` : emits anomaly events only, no impact on agent execution
- `warn` : same as observe + prints formatted warning to terminal for high/critical anomalies
- `halt` : on critical severity, sends halt signal to main process, raises exception in agent execution thread

Failure Explanation

On any `ERROR` trace event, sidecar automatically generates:

- `root_cause` : one sentence identifying the fundamental cause
- `contributing_factors` : array of secondary causes

- `first_bad_step` : step_id of earliest step where things went wrong
- `suggested_fix` : concrete actionable recommendation

PHASE 6 — Policy as Code

Overview

Define agent behavior rules in declarative YAML files stored in version control. Enforced consistently across environments. The enterprise compliance feature.

Policy File Format

Default file: `interveil.policy.yaml`

```yaml
version: "1.0"
agents:
  - id: "customer-service-agent"
    rules:
      - id: "no-production-writes"
        condition:
          tool: "database_query"
          input.database: "production"
          input.readonly: false
        action: block
        severity: critical
        message: "Agent attempted production write — blocked by policy"

      - id: "require-approval-for-refunds"
        condition:
          tool: "process_refund"
          input.amount:
            greaterThan: 100
        action: require_approval
        severity: high

environments:
  production:
    strictMode: true
    haltOnCritical: true
  staging:
    strictMode: false
```

```
  haltOnCritical: false
```

Actions

- `block` : prevent execution, emit `POLICY_VIOLATION` trace event
- `allow` : explicitly allow
- `allow_and_audit` : allow but emit `POLICY_AUDIT` trace event
- `require_approval` : pause, prompt developer in terminal for y/n
- `halt_agent` : immediately terminate agent process

CI/CD Integration

- `interveil validate-policy` — validates all policy files, exit code 0/1
- `interveil lint-policy` — checks for unreachable rules, conflicts, missing descriptions

---

PHASE 7 — Multi-Agent Orchestration Visibility

Overview

Extend Interveil to handle systems where one orchestrator agent spawns and directs multiple sub-agents. Visualize the full call graph, trace context propagation, and enforce access control across the agent hierarchy.

Trace Context Propagation

```
// Parent agent
const ctx = Interveil.getTraceContext()
const childResult = await spawnChildAgent(task, { interveilContext: ctx })

// Child agent
const tracedChild = trace(childAgent, { parentContext: options.interveilContext }
```

Graph Data Model

```
{
  session_id: string,
  parent_session_id: string | null,
  agent_id: string,
  depth: number,
  spawned_at_step_id: string
}
```

Graph View UI

- “Orchestration” tab in trace viewer
- DAG visualization of all agents in current run
- Node color coding: green (success), red (failed), yellow (blocked), blue (running)
- Click node to open full trace in main panel

Context Pollution Detection

- `CONTEXT_LEAK` : child agent reasoning references parent’s private context
- `CONTEXT_CONTAMINATION` : orchestrator influenced by erroneous sub-agent output without acknowledging failure

Cross-Agent Permission Inheritance

```
agents:
  - id: "orchestrator"
    rules:
      - tool: "production_database"
        action: block
    childPolicy:
      inherit: true
      additionalRestrictions:
        - tool: "send_email"
          action: block
```

A child agent can never have more permissions than its parent. Enforced at wrapper and MCP proxy level.

PHASE 8 — Teams + Collaboration

Overview

Transform Interveil from a local developer tool into a shared team platform.

Authentication

- Auth enabled when `AUTH_ENABLED=true` env var is set
- Providers: local username/password, GitHub OAuth, Google OAuth
- JWT tokens issued on login

Roles

- `viewer` : read-only access to all sessions and traces
- `developer` : can run agents, create/label sessions, add comments
- `admin` : full access including policy management and user management

Collaborative Trace Review

- Any session can be opened in review mode
- Comments anchored to specific trace steps
- @mentions notify team members
- Unresolved comments shown as badge on session in history list

Multi-User Deployment

- Team mode uses PostgreSQL instead of SQLite
- `DATABASE_URL` environment variable configures connection
- Docker Compose updated to include `postgres` service

PHASE 9 — Integrations + Ecosystem

Overview

Native SDKs for major agent frameworks, CI/CD integration, and a full REST API.

Native Framework SDKs

LangChain

```
import { InterveillCallbackHandler } from 'interveil/langchain'
const handler = new InterveillCallbackHandler()
const chain = new AgentExecutor({ callbacks: [handler] })
```

Anthropic SDK

```
import { createTracedAnthropic } from 'interveil/anthropic'
const anthropic = createTracedAnthropic(new Anthropic())
```

OpenAI Agents SDK

```
import { tracedRunner } from 'interveil/openai-agents'
const result = await tracedRunner.run(agent, input)
```

Python Frameworks (CrewAI, AutoGen)

- `pip install interveil` — hook-based integration, sends events via HTTP POST

REST API

Base URL: `http://localhost:3000/api/v1`

Key endpoints:

- `GET /sessions` — list with filter params
- `GET /sessions/:id` — full session detail
- `GET /sessions/:id/events` — all trace events
- `POST /sessions/:id/labels` — add label
- `GET /policies` — list all policies

- `PUT /policies/:id` — update policy
- `GET /metrics/cost` — cost tracking data
- `GET /metrics/anomalies` — sidecar anomaly summary

Authenticated via `X-Interveil-Key` header.

CI/CD Integration

GitHub Actions

```
- name: Run agent test suite
  uses: interveil/github-action@v1
  with:
    command: npm run test:agents
    policy: interveil.policy.yaml
    failOnViolation: true
    failOnAnomaly: critical
```

JUnit XML Export

- `interveil export --format junit --session [id]` produces JUnit XML
- Trace results appear in any CI system's test results UI

VSCode Extension Full Enhancement (from Phase 1.75)

- Inline trace annotations on hover
- "Run with Interveil" CodeLens above agent entry points
- Policy violations appear as VSCode warnings/errors
- Sidecar fix suggestions appear as VSCode code actions